

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BỘ MÔN THỰC TẬP CƠ SỞ



BÁO CÁO TỔNG HỢP
ĐỀ TÀI: HỆ THỐNG GỌI Ý PHIM

Giảng viên hướng dẫn:

Đào Thị Thúy Quỳnh

Họ và tên sinh viên:

Nguyễn Văn Quân - B22DCCN666

Võ Thanh Huyền - B22DCCN403

Trần Hải Hưng - B19DCCN733

Nguyễn Thành Lâm - B20DCCN392

Nhóm thực hiện:

05

Hà Nội – 2026

PHẦN 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ PHÂN TÍCH HỆ THỐNG

1.1. Lý do chọn đề tài

Trong kỷ nguyên bùng nổ của ngành công nghiệp điện ảnh, số lượng phim được sản xuất và phát hành trên các nền tảng trực tuyến là vô cùng lớn. Điều này gây khó khăn cho người dùng trong việc tìm kiếm và lựa chọn những bộ phim thực sự phù hợp với sở thích cá nhân. Hệ thống gợi ý phim (Movie Recommendation System) ra đời như một giải pháp tối ưu, không chỉ giúp nâng cao trải nghiệm giải trí của người xem mà còn tăng khả năng tiếp cận các nội dung tiềm năng trên các nền tảng số.

Dự án "End-to-End Movie Recommendation System" được nhóm lựa chọn nhằm xây dựng một hệ thống hoàn chỉnh từ khâu xử lý dữ liệu thô, huấn luyện mô hình máy học đến triển khai ứng dụng web thực tế.

1.2. Mục tiêu và Phạm vi dự án

- Mục tiêu chính: Xây dựng một công cụ gợi ý phim cá nhân hóa mạnh mẽ, có khả năng phân tích nội dung phim để đưa ra các đề xuất tương đồng nhất với một bộ phim cụ thể mà người dùng yêu thích.
- Mục tiêu kỹ thuật:
 - Xây dựng quy trình xử lý dữ liệu văn bản chuyên sâu (NLP).
 - Áp dụng các thuật toán máy học để tính toán độ tương đồng giữa hàng ngàn bộ phim.
 - Phát triển giao diện người dùng trực quan, tích hợp hình ảnh poster từ API bên ngoài.
 - Đảm bảo quy trình vận hành và triển khai (DevOps) đạt chuẩn chuyên nghiệp.

1.3. Công nghệ và Thư viện sử dụng (Tech Stack)

Hệ thống được phát triển dựa trên ngôn ngữ lập trình Python cùng các công cụ hiện đại:

- Xử lý dữ liệu & Tính toán: Pandas (quản lý DataFrame), Numpy (tính toán ma trận), Scipy.
- Học máy (Machine Learning): Scikit-learn (phiên bản 1.2.2) – sử dụng để vector hóa văn bản và tính toán độ tương đồng Cosine.
- Xử lý ngôn ngữ tự nhiên (NLP): NLTK (Natural Language Toolkit) – dùng để tách gốc từ (Stemming) và làm sạch dữ liệu văn bản.
- Giao diện & Web Framework: Streamlit (hoặc Flask) – xây dựng dashboard tương tác và hiển thị kết quả.
- Kết nối dữ liệu: tmdbv3api và thư viện requests để truy xuất thông tin, poster phim từ API của The Movie Database (TMDB).

- Quản lý dự án (DevOps): DVC (Data Version Control) để quản lý phiên bản dữ liệu và Git LFS để lưu trữ các tệp mô hình có dung lượng lớn

1.4. Phân tích chi tiết dữ liệu đầu vào

Hệ thống sử dụng bộ dữ liệu TMDB 5000, một tập hợp dữ liệu phong phú về điện ảnh được trích xuất từ Kaggle. Quá trình phân tích bao gồm các bước xử lý dữ liệu thô phức tạp:

- Cấu trúc tệp tin nguồn:
 - `tmdb_5000_movies.csv`: Chứa các thông số định tính và định lượng của 4803 bộ phim như ngân sách (budget), thể loại (genres), từ khóa (keywords), tóm tắt (overview), và điểm đánh giá.
 - `tmdb_5000_credits.csv`: Tập trung vào nhân sự, bao gồm danh sách đầy đủ các diễn viên (cast) và đội ngũ sản xuất (crew).
- Quy trình hợp nhất và tinh lọc:
 - Hai tập dữ liệu được kết hợp bằng hàm `merge()` dựa trên cột định danh chung là `title` để tạo ra một bảng dữ liệu tổng thể.
 - Loại bỏ nhiễu: Các thuộc tính không hỗ trợ cho việc gợi ý nội dung như `homepage`, `status`, `runtime`, `budget` đã bị loại bỏ để tối ưu hóa bộ nhớ.
 - Trích xuất đặc trưng (Feature Selection): Hệ thống chỉ giữ lại 7 cột quan trọng nhất: `movie_id`, `title`, `overview`, `genres`, `keywords`, `cast`, và `crew`.
- Thách thức về định dạng dữ liệu:
 - Dữ liệu trong các cột như `genres`, `keywords`, `cast` ban đầu nằm dưới dạng chuỗi (String) chứa cấu trúc JSON phức tạp.
 - Giải pháp: Sử dụng thư viện `ast.literal_eval` để chuyển đổi các chuỗi này thành đối tượng Python (List/Dictionary). Từ đó, hệ thống trích xuất tên thể loại, các từ khóa chính, Top 3 diễn viên nổi bật nhất và duy nhất 1 Đạo diễn (lọc theo `job == 'Director'`).

1.5. Kiến trúc thuật toán lõi và Quy trình xử lý

Hệ thống vận hành dựa trên kiến trúc Content-based Filtering (Gợi ý dựa trên nội dung), biến các thuộc tính văn bản thành không gian vector toán học để tính toán độ tương đồng.

1.5.1. Tiền xử lý ngôn ngữ tự nhiên (NLP Pipeline)

Để máy tính có thể "hiểu" được nội dung phim, dữ liệu văn bản phải trải qua quy trình chuẩn hóa nghiêm ngặt:

- Loại bỏ khoảng trắng (Space Removal): Chuyển đổi các thực thể như "Science Fiction" thành "ScienceFiction" hoặc "Johnny Depp" thành "JohnnyDepp". Bước này cực kỳ quan trọng để mô hình vector hóa hiểu đây là một từ đơn nhất, tránh nhầm lẫn giữa các diễn viên hoặc thể loại có tên trùng lặp một phần.
- Stemming (Tách gốc từ): Sử dụng lớp PorterStemmer từ thư viện NLTK. Kỹ thuật này đưa các từ có cùng gốc về một dạng duy nhất (ví dụ: "actors", "acting" đều về "actor"), giúp giảm thiểu sự bùng nổ số lượng từ vựng và tăng độ chính xác khi so khớp.
- Hợp nhất Tags: Toàn bộ nội dung từ cốt truyện, thể loại đến tên diễn viên/đạo diễn được cộng dồn vào một trường dữ liệu duy nhất gọi là "tags". Mỗi bộ phim lúc này được đại diện bởi một đoạn văn bản tóm gọn toàn bộ đặc trưng của nó.

1.5.2. Vector hóa và Tính toán độ tương đồng

- Kỹ thuật Bag of Words: Sử dụng CountVectorizer với tham số `max_features=5000` để trích xuất 5000 từ khóa xuất hiện nhiều nhất trong toàn bộ 4803 bộ phim. Hệ thống cũng tự động loại bỏ các "stop words" (từ dừng) tiếng Anh không mang ý nghĩa phân loại như "and", "the", "is".
- Ma trận hóa: Kết quả là một ma trận thưa kích thước 4803 x 5000, trong đó mỗi bộ phim là một điểm (vector) trong không gian 5000 chiều.
- Cosine Similarity (Độ tương đồng Cosine):
 - Thay vì dùng khoảng cách Euclid (đo khoảng cách thẳng), hệ thống sử dụng Cosine Similarity để đo góc giữa hai vector phim.
 - Trong không gian nhiều chiều, việc đo góc hiệu quả hơn vì nó tập trung vào hướng của dữ liệu (nội dung) hơn là độ dài của văn bản. Giá trị tương đồng dao động từ 0 đến 1; giá trị càng gần 1, hai bộ phim càng giống nhau.

1.5.3. Logic gợi ý (Recommendation Logic)

Khi người dùng nhập một bộ phim:

1. Hệ thống xác định chỉ số (index) của phim đó trong DataFrame.
2. Truy cập hàng tương ứng trong ma trận tương đồng (Similarity Matrix).
3. Sắp xếp các phim khác theo điểm số tương đồng giảm dần.
4. Trích xuất 5 phim có điểm số cao nhất (loại trừ chính nó) để hiển thị lên giao diện.

Toàn bộ mô hình và dữ liệu sau khi huấn luyện được đóng gói vào các file .pkl (như `movies.pkl`, `similarity.pkl`) bằng thư viện pickle để sẵn sàng cho việc triển khai ứng dụng Web.

PHẦN 2: QUY TRÌNH TRIỂN KHAI DỰ ÁN

Quá trình thực hiện dự án được chia thành 3 giai đoạn chính, kéo dài trong 6 tuần làm việc tập trung:

2.1. Giai đoạn 1: Xử lý dữ liệu và Trích xuất đặc trưng (Tuần 1 & Tuần 2)

Đây là giai đoạn nền tảng, quyết định chất lượng đầu vào cho các thuật toán gợi ý. Mục tiêu chính là chuyển đổi dữ liệu thô từ định dạng bảng phức tạp sang một cấu trúc tinh gọn, sẵn sàng cho việc tính toán toán học.

2.1.1. Tuần 1: Thiết lập môi trường và Tiền xử lý sơ bộ

Trong tuần đầu tiên, nhóm tập trung vào việc hình thành khung làm việc và làm sạch dữ liệu ở mức độ cơ bản.

- Thiết lập môi trường phát triển:
 - Khởi tạo môi trường ảo (Virtual Environment) để quản lý độc lập các thư viện, tránh xung đột phiên bản.
 - Cài đặt các thư viện lõi bao gồm: pandas và numpy để xử lý ma trận dữ liệu; matplotlib và seaborn phục vụ phân tích trực quan (EDA); scikit-learn cho các bước vector hóa và pickle để đóng gói mô hình sau này.
- Thu thập và Hợp nhất dữ liệu (Data Merging):
 - Hệ thống nạp dữ liệu từ hai nguồn chính: tmdb_5000_movies.csv (chứa thông tin phim) và tmdb_5000_credits.csv (chứa thông tin nhân sự).
 - Sử dụng hàm merge() để kết hợp hai tập dữ liệu dựa trên cột title (hoặc id), tạo thành một DataFrame thống nhất chứa thông tin của 4803 bộ phim.
- Lọc bỏ thuộc tính dư thừa:
 - Nhóm tiến hành đánh giá và chỉ giữ lại 7 cột quan trọng nhất cho việc gợi ý nội dung: movie_id, title, overview, genres, keywords, cast, và crew.
 - Các thông tin không hỗ trợ trực tiếp cho thuật toán như homepage, status, runtime, budget được loại bỏ để tối ưu bộ nhớ RAM.
- Xử lý giá trị thiếu và dữ liệu trùng lặp:
 - Kiểm tra và loại bỏ các dòng có cột overview bị trống (chiếm tỉ lệ rất nhỏ, khoảng 3 phim) để đảm bảo không gây lỗi khi xử lý ngôn ngữ tự nhiên sau này.

2.1.2. Tuần 2: Tiền xử lý dữ liệu chuyên sâu và Xây dựng "Tags"

Tuần thứ hai tập trung vào việc trích xuất các đặc trưng định tính từ cấu trúc dữ liệu phức tạp.

- Chuyển đổi định dạng dữ liệu phức tạp:

- Sử dụng thư viện `ast.literal_eval` để chuyển đổi các cột dạng chuỗi JSON (genres, keywords, cast, crew) thành danh sách (List) trong Python.
- Xây dựng các hàm hỗ trợ (Helper functions) để trích xuất thông tin cụ thể: lấy tên các thể loại phim, chọn lọc Top 3 diễn viên nổi bật nhất từ cột cast và trích xuất duy nhất tên Đạo diễn (Director) từ cột crew.
- Chuẩn hóa từ khóa và loại bỏ khoảng trắng:
 - Thực hiện kỹ thuật loại bỏ khoảng trắng giữa các cụm từ (ví dụ: "Johnny Depp" thành "JohnnyDepp", "Science Fiction" thành "ScienceFiction").
 - Mục đích: Việc này giúp mô hình vector hóa hiểu đây là một thực thể duy nhất, tránh nhầm lẫn giữa các cá nhân hoặc thể loại có cùng tên riêng nhưng khác biệt về ngữ cảnh.
- Khởi tạo trường dữ liệu "Tags":
 - Hệ thống tiến hành hợp nhất toàn bộ nội dung từ các cột: overview, genres, keywords, cast, và crew thành một trường dữ liệu duy nhất gọi là "tags".
 - Mỗi bộ phim lúc này được đại diện bởi một đoạn văn bản tổng hợp chứa đựng toàn bộ đặc trưng đặc sắc nhất của nó.
- Xử lý ngôn ngữ tự nhiên - Stemming:
 - Sử dụng thư viện `nlk` với lớp `PorterStemmer` để đưa các từ về gốc từ ban đầu (ví dụ: "actors", "acting" đều về "actor").
 - Kỹ thuật này giúp giảm bớt số lượng từ vựng dư thừa, làm sạch dữ liệu và tăng đáng kể độ chính xác khi tính toán độ tương đồng giữa các phim ở giai đoạn sau.

2.1.3. Tổng kết vấn đề và giải pháp giai đoạn 1

- Thách thức: Dữ liệu thô nằm dưới dạng chuỗi phức tạp và có nhiều giá trị nhiễu.
- Giải pháp: Nhóm đã sử dụng `ast` để chuyển đổi an toàn, áp dụng hàm `lambda` kết hợp với `apply()` của `Pandas` để tăng tốc độ xử lý trên 5000 dòng dữ liệu, đồng thời thêm các câu lệnh điều kiện `if-else` để kiểm tra dữ liệu null, tránh crash chương trình

2.2. Giai đoạn 2: Xây dựng thuật toán và Phát triển giao diện (Tuần 3 & Tuần 4)

Sau khi dữ liệu đã được làm sạch và chuẩn hóa ở Giai đoạn 1, nhóm tiến hành chuyển đổi các đặc trưng văn bản thành mô hình toán học và xây dựng nền tảng tương tác cho người dùng.

2.2.1. Tuần 3: Vector hóa văn bản và Xây dựng thuật toán gợi ý

Mục tiêu cốt lõi của tuần này là biến trường dữ liệu "tags" (văn bản) thành các vector số học để máy tính có thể tính toán mức độ tương đồng.

- Vector hóa bằng kỹ thuật Bag of Words:
 1. Sử dụng lớp CountVectorizer từ thư viện Scikit-learn để chuyển đổi 4803 bộ phim thành một ma trận vector kích thước 4803 x 5000.
 2. Tham số tối ưu: Thiết lập max_features=5000 để chỉ chọn ra 5000 từ vựng xuất hiện nhiều nhất và có ý nghĩa đặc trưng cao nhất, đồng thời loại bỏ các "stop words" tiếng Anh (như the, is, and...).
- Tính toán độ tương đồng (Similarity Scoring):
 1. Nhóm quyết định sử dụng Cosine Similarity thay vì khoảng cách Euclid. Lý do là trong không gian cao chiều (5000 chiều), việc đo góc giữa các vector sẽ phản ánh chính xác hơn độ tương đồng về nội dung và thể loại phim.
 2. Kết quả thu được là một ma trận vuông 4803x4803, trong đó mỗi ô [i, j] thể hiện điểm số tương đồng (từ 0 đến 1) giữa phim thứ i và phim thứ j.
- Phát triển logic hàm gợi ý (recommend):
 1. Tìm chỉ số (index) của phim đầu vào trong cơ sở dữ liệu.
 2. Truy cập hàng tương ứng trong ma trận tương đồng để lấy danh sách điểm số của các phim khác.
 3. Sử dụng hàm enumerate() để cố định chỉ số gốc của từng phim, sau đó sắp xếp theo thứ tự điểm số giảm dần.
 4. Trích xuất 5 bộ phim có điểm số cao nhất (không tính chính nó) để đưa ra đề xuất.
- Đóng gói mô hình: Sử dụng thư viện pickle để xuất (export) DataFrame phim và ma trận tương đồng thành các tệp .pkl nhằm phục vụ việc nạp dữ liệu nhanh chóng cho giao diện web.

2.2.2. Tuần 4: Phát triển giao diện người dùng và Tích hợp API

Mục tiêu là tạo ra một ứng dụng Web trực quan, giúp người dùng không cần thao tác với code mà vẫn nhận được kết quả gợi ý chuyên nghiệp.

- Xây dựng khung ứng dụng bằng Streamlit:
 - Thiết kế giao diện trực quan với tiêu đề "Movie Recommender System" và một hộp chọn (Selectbox/Dropdown) chứa danh sách gần 5000 tên phim từ tập dữ liệu.
 - Thiết lập nút bấm "Recommend" để kích hoạt quy trình tính toán và hiển thị.
- Tích hợp hình ảnh qua TMDB API:

- Để tăng trải nghiệm người dùng, nhóm đăng ký khóa truy cập (API Key) từ hệ thống The Movie Database (TMDb).
- Viết hàm `fetch_poster(movie_id)`: Hàm này sử dụng thư viện `requests` để gửi yêu cầu đến máy chủ TMDb, nhận về tệp JSON chứa đường dẫn ảnh (`poster_path`) và ghép nối với URL cơ sở để lấy ảnh poster chất lượng cao.
- Thiết kế bố cục hiển thị (Layout Design):
 - Sử dụng hàm `st.columns(5)` của Streamlit để chia màn hình thành 5 cột theo hàng ngang.
 - Mỗi cột sẽ hiển thị song song ảnh Poster phim bên trên và tên phim tương ứng bên dưới, tạo cảm giác chuyên nghiệp như các nền tảng xem phim trực tuyến hiện nay.

2.2.3. Vấn đề kỹ thuật và Giải pháp giai đoạn 2

- Thách thức về tài nguyên: Ma trận tương đồng 4803×4803 tiêu tốn rất nhiều bộ nhớ RAM, dễ gây đứng máy khi tính toán.
 - Giải pháp: Sử dụng kiểu dữ liệu `float32` cho ma trận để tiết kiệm dung lượng lưu trữ mà vẫn đảm bảo độ chính xác của điểm số.
- Thách thức về tốc độ mạng: Việc gọi API lấy ảnh liên tục có thể gây trễ (latency) khi hiển thị.
 - Giải pháp: Áp dụng Decorator `@st.cache_data` của Streamlit để lưu trữ tạm thời các kết quả API đã tải, giúp các lần tìm kiếm sau phản hồi ngay lập tức mà không cần tải lại ảnh cũ.

2.3. Giai đoạn 3: Tối ưu hóa hệ thống và Quy trình DevOps (Tuần 5 & Tuần 6)

Sau khi hoàn thiện thuật toán và giao diện, nhóm tập trung vào việc chuẩn hóa mã nguồn, quản lý tài nguyên và triển khai ứng dụng lên môi trường điện toán đám mây.

2.3.1. Tuần 5: Chuẩn hóa cấu trúc mã nguồn và Quản lý tệp tin lớn

Mục tiêu tuần này là tái cấu trúc (Refactoring) mã nguồn và thiết lập quy trình quản lý phiên bản chuyên nghiệp.

- Chuẩn hóa cấu trúc thư mục:
 - Nhóm chuyển đổi từ dạng script rời rạc sang cấu trúc dự án chuẩn để dễ dàng bảo trì.
 - Cấu trúc bao gồm: thư mục `src/` (chứa mã nguồn xử lý), `models/` (lưu trữ tệp `.pkl`), `static/` và `templates/` (cho giao diện), cùng tệp chạy chính `app.py` và tệp quản lý thư viện `requirements.txt`.
- Quản lý tệp tin lớn (Git LFS & DVC):

- Thách thức: GitHub giới hạn kích thước tệp tin dưới 100MB, trong khi ma trận tương đồng similarity.pkl thường vượt ngưỡng này.
- Giải pháp: Nhóm cài đặt và cấu hình Git LFS (Large File Storage) để theo dõi các tệp .pkl. Ngoài ra, hệ thống còn tích hợp DVC (Data Version Control) để quản lý các phiên bản dữ liệu và mô hình một cách độc lập với mã nguồn.
- Tối ưu hóa hiệu suất ứng dụng:
 - Sử dụng Decorator `@st.cache_resource` (trong Streamlit) cho hàm tải mô hình.
 - Mục đích: Đảm bảo ma trận tương đồng chỉ được nạp vào bộ nhớ RAM một lần duy nhất khi khởi động ứng dụng, giúp tiết kiệm tài nguyên hệ thống khi có nhiều người dùng truy cập đồng thời.
- Xây dựng tài liệu dự án: Hoàn thiện tệp README.md chuyên nghiệp, bao gồm hướng dẫn cài đặt, cách lấy API Key và mô tả ngắn gọn về thuật toán để hỗ trợ cộng đồng mã nguồn mở.

2.3.2. Tuần 6: Triển khai Cloud và Bảo mật hệ thống

Tuần cuối cùng tập trung vào việc đưa ứng dụng lên internet và kiểm thử cuối cùng trên môi trường thực tế.

- Triển khai trên nền tảng Cloud:
 - Nhóm thực hiện đồng bộ kho lưu trữ GitHub với Streamlit Cloud (hoặc Render) để triển khai ứng dụng trực tuyến qua URL công khai.
 - Thiết lập tệp Dockerfile để đóng gói ứng dụng, đảm bảo tính nhất quán giữa môi trường lập trình cục bộ và môi trường máy chủ.
- Quản lý biến môi trường và Bảo mật:
 - Áp dụng quy tắc bảo mật không để trực tiếp API Key của TMDb trong mã nguồn.
 - Sử dụng tính năng "Secrets" trên Dashboard của nền tảng triển khai để lưu trữ các khóa bí mật một cách an toàn.
- Xử lý lỗi ngoại lệ và Kiểm thử thực tế:
 - Bổ sung các khối lệnh try-except để xử lý các tình huống mất kết nối mạng khi gọi API hoặc khi người dùng tìm kiếm những phim không tồn tại trong cơ sở dữ liệu.
 - Kiểm tra tính tương thích của giao diện trên nhiều thiết bị và trình duyệt khác nhau để đảm bảo trải nghiệm người dùng đồng nhất.

2.3.3. Tổng kết vấn đề và Giải pháp giai đoạn cuối

- Vấn đề gặp phải:

- Lỗi "Out of Memory" (OOM): Máy chủ miễn phí thường giới hạn RAM (khoảng 1GB), gây sập ứng dụng khi nạp ma trận tương đồng quá lớn.
- Xung đột phiên bản: Một số thư viện trên server tự động cập nhật gây lỗi code cũ.
- Hướng giải quyết:
 - Nén dữ liệu: Sử dụng các thư viện như bz2 hoặc gzip để nén tệp .pkl, giúp giảm 50-70% kích thước tệp tin trước khi đẩy lên server.
 - Cố định phiên bản: Quy định chính xác phiên bản các thư viện trong requirements.txt (ví dụ: scikit-learn==1.2.2, pandas==1.5.3) để tránh sai lệch môi trường.

PHẦN 3: ĐÁNH GIÁ KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN

3.1. Kết quả đạt được (Achievements)

Sau 6 tuần triển khai, dự án đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu, tạo ra một sản phẩm hoàn thiện cả về mặt thuật toán lẫn ứng dụng thực tế:

- Sản phẩm hoàn thiện (End-to-End Product): Xây dựng thành công ứng dụng Web cho phép người dùng tìm kiếm và nhận gợi ý phim trực tuyến 24/7 thông qua nền tảng Streamlit Cloud.
- Độ chính xác của mô hình: Hàm recommend() hoạt động ổn định và chính xác. Qua thử nghiệm với các phim nổi tiếng như Avatar hay The Dark Knight, hệ thống đã trả về danh sách các phim có sự tương đồng rõ rệt về thể loại, dàn diễn viên và không khí phim.
- Trải nghiệm người dùng (UX): Việc tích hợp thành công TMDB API để hiển thị Poster phim thay vì chỉ liệt kê văn bản thuần túy đã giúp tăng tính thẩm mỹ và mức độ tương tác của người dùng một cách đáng kể.
- Tối ưu hóa kỹ thuật: Hệ thống phản hồi kết quả gần như tức thì nhờ áp dụng kỹ thuật caching dữ liệu (@st.cache_data, @st.cache_resource), giúp giảm tải cho server và rút ngắn thời gian chờ đợi của người dùng.

3.2. Đánh giá về mặt kỹ thuật (Technical Evaluation)

- Quản lý dữ liệu: Nhóm đã thành thạo quy trình tiền xử lý dữ liệu phức tạp (NLP), từ việc chuyển đổi cấu trúc JSON đến kỹ thuật tách gốc từ (Stemming) để làm gọn tập dữ liệu 4803 bộ phim.
- Hiệu quả thuật toán: Việc lựa chọn Cosine Similarity thay vì khoảng cách Euclid được chứng minh là đúng đắn, giúp đo lường chính xác sự tương đồng trong không gian 5000 chiều của các vector từ khóa.

- Năng lực triển khai (DevOps): Dự án đã giải quyết thành công bài toán lưu trữ tệp tin lớn bằng Git LFS và nén ma trận tương đồng (giảm từ ~200MB xuống dưới 50MB), giúp vượt qua giới hạn bộ nhớ của các máy chủ Cloud miễn phí.

3.3. Hạn chế và Thách thức (Limitations & Challenges)

Mặc dù đạt được kết quả khả quan, dự án vẫn còn một số điểm cần cải thiện:

- Giới hạn tài nguyên Cloud: Các server miễn phí (như Streamlit Cloud) thường giới hạn RAM ở mức 1GB, gây ra nguy cơ lỗi "Out of Memory" (OOM) nếu cơ sở dữ liệu phim tiếp tục mở rộng.
- Độ trễ API: Tốc độ tải ảnh Poster đôi khi bị ảnh hưởng bởi đường truyền mạng và giới hạn yêu cầu (API Limit) từ phía nhà cung cấp TMDB.
- Tính đơn nhất của thuật toán: Hiện tại hệ thống chỉ mới sử dụng Content-based Filtering (gợi ý dựa trên nội dung phim), chưa tính đến yếu tố sở thích của các người dùng tương đồng (Collaborative Filtering).

3.4. Hướng phát triển tương lai (Future Work)

Để nâng cấp hệ thống trở nên chuyên nghiệp hơn, nhóm đề xuất các hướng phát triển sau:

- Xây dựng mô hình Hybrid: Kết hợp giữa Content-based và Collaborative Filtering để đưa ra các gợi ý đa dạng và mang tính cá nhân hóa sâu sắc hơn cho từng người dùng.
- Nâng cấp cơ sở dữ liệu: Mở rộng tập dữ liệu lên hàng chục ngàn phim và sử dụng các hệ quản trị cơ sở dữ liệu chuyên dụng như PostgreSQL hoặc MongoDB để quản lý thay vì dùng file .pkl.
- Tính năng tương tác nâng cao: Bổ sung tính năng cho phép người dùng đăng nhập, đánh giá phim (Rating) và lưu danh sách phim yêu thích (Watchlist) để hệ thống tự động học hỏi sở thích theo thời gian thực.
- Docker hóa toàn diện: Hoàn thiện tệp Dockerfile để cho phép triển khai ứng dụng một cách dễ dàng trên mọi môi trường máy chủ khác nhau (On-premise hoặc Multi-cloud).

PHẦN 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết luận chung

Trải qua 6 tuần thực hiện dự án thực tập cơ sở, nhóm đã hoàn thành mục tiêu xây dựng một hệ thống gợi ý phim toàn diện (End-to-End) từ khâu nghiên cứu

dữ liệu đến triển khai ứng dụng thực tế. Dự án không chỉ là một bài tập về lập trình mà còn là quá trình làm chủ quy trình phát triển phần mềm hiện đại, kết hợp giữa Khoa học dữ liệu (Data Science) và Kỹ thuật phần mềm (Software Engineering).

Các thành tựu chính đã đạt được:

- Về mặt kỹ thuật: Nhóm đã làm chủ được quy trình tiền xử lý ngôn ngữ tự nhiên (NLP) phức tạp, bao gồm kỹ thuật Stemming bằng NLTK và vector hóa văn bản bằng CountVectorizer để chuyển đổi nội dung phim thành các đại diện toán học. Việc áp dụng thuật toán Cosine Similarity đã mang lại kết quả gợi ý có độ chính xác cao về mặt nội dung.
- Về mặt ứng dụng: Hệ thống đã được triển khai thành công lên nền tảng đám mây (Cloud), cho phép người dùng truy cập trực tuyến 24/7 với giao diện trực quan, hiển thị Poster phim thời gian thực thông qua kết nối TMDb API.
- Về mặt quy trình (DevOps): Nhóm đã vượt qua các thách thức về giới hạn tài nguyên máy chủ bằng cách sử dụng Git LFS để quản lý tệp tin lớn và áp dụng các kỹ thuật nén dữ liệu, caching mô hình để tối ưu hóa bộ nhớ RAM.

4.2. Hướng phát triển và mở rộng

Mặc dù sản phẩm hiện tại đã đáp ứng tốt các yêu cầu cơ bản, nhóm xác định một số hướng phát triển quan trọng để nâng cấp hệ thống trong tương lai:

- Chuyển đổi sang mô hình Hybrid Recommendation:
 - Hệ thống hiện tại chỉ dựa trên nội dung (Content-based Filtering). Hướng phát triển tiếp theo là tích hợp thêm thuật toán Collaborative Filtering (Gợi ý dựa trên hành vi người dùng tương đồng) để mang lại kết quả đa dạng và khách quan hơn.
- Tối ưu hóa quản lý dữ liệu:
 - Thay thế việc lưu trữ bằng các file .pkl (vốn tiêu tốn nhiều RAM và dễ gây lỗi Out of Memory) bằng các hệ quản trị cơ sở dữ liệu chuyên dụng như PostgreSQL hoặc MongoDB.
 - Thực hiện kỹ thuật chỉ lưu trữ Top 10-20 phim tương đồng nhất thay vì lưu toàn bộ ma trận tương đồng để giảm kích thước file mô hình từ ~200MB xuống dưới 50MB.
- Phát triển tính năng người dùng cá nhân:
 - Xây dựng hệ thống đăng nhập, cho phép người dùng lưu lại danh sách phim yêu thích (Watchlist) và thực hiện đánh giá (Rating) trực tiếp trên giao diện.
 - Tích hợp hệ thống theo dõi lịch sử tìm kiếm để cải thiện độ chính xác của các đề xuất theo thời gian.
- Tự động hóa và Mở rộng quy mô:

- Hoàn thiện quy trình CI/CD và đóng gói ứng dụng bằng Docker để đảm bảo tính nhất quán khi triển khai trên các môi trường máy chủ khác nhau (On-premise hoặc Multi-cloud).
- Nâng cấp lên bộ dữ liệu lớn hơn (ví dụ: TMDB full dataset thay vì bản 5000 phim) để mở rộng phạm vi gợi ý